

Credit Card Fraud Detection

What is credit card fraud ? The model we use: Decision Tree

Credit card fraud is when someone uses a credit card or its details without permission to steal money or buy things.

Common examples:

- stealing a physical card and using it
- using the card number online after it was leaked or copied
- creating fake cards with stolen data
- tricking someone into giving card details through phishing or scams

Why Decision Tree ?

1) Easy to understand and explain

Decision trees show clearly how a decision is made. For example, the model might check the transaction amount, then the location, then the time. This makes it easier for banks and analysts to understand why a transaction was flagged as fraud.

2) Works well with different types of transaction data

Credit card fraud detection uses many kinds of data, such as amount, time, merchant type, country, and device. Decision trees can them

Credit Card Fraud Detection

Python Implementation

```
# 1) Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    accuracy_score
)

# 2) Load the dataset
df = pd.read_csv("creditcard.csv")
```

`from sklearn.model_selection import train_test_split`

is used to split the data to training and testing data

`DecisionTreeClassifier` creates the decision tree model

`plot_tree` draws the tree

Credit Card Fraud Detection

Checking how the data is distributed

```
# 4) Check class distribution
print(df["Class"].value_counts())

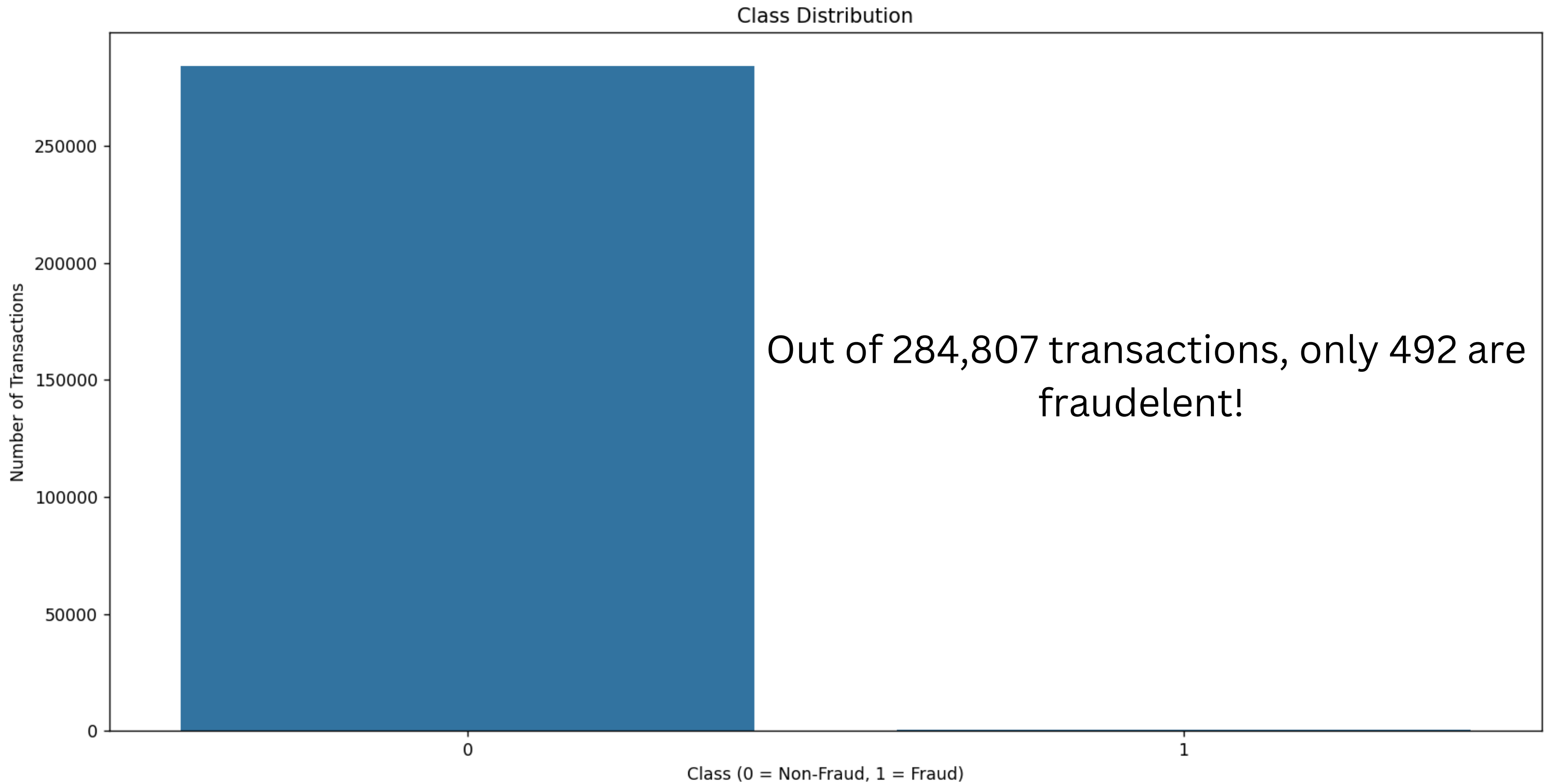
# Total transactions
total_transactions = len(df)

# Total fraud and non-fraud
fraud_count = df["Class"].sum()
non_fraud_count = total_transactions - fraud_count

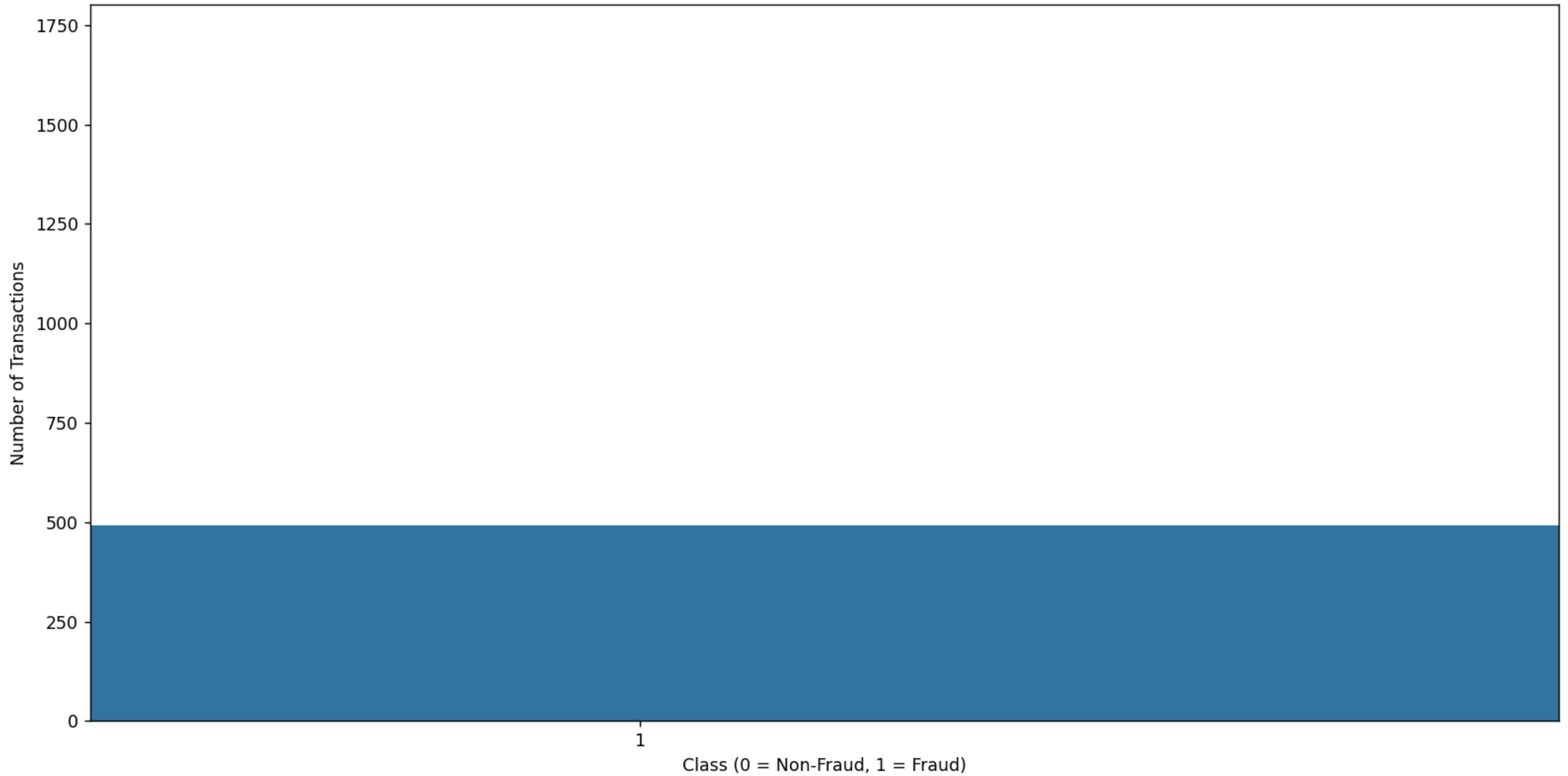
# Percentages
fraud_percentage = (fraud_count / total_transactions) * 100
non_fraud_percentage = (non_fraud_count / total_transactions) * 100
```

Visualizing the data

```
# 5) Visualize class distribution
sns.countplot(x="Class", data=df)
plt.title("Class Distribution")
plt.xlabel("Class (0 = Non-Fraud, 1 = Fraud)")
plt.ylabel("Number of Transactions")
plt.show()
```



Class Distribution



Credit Card Fraud Detection

Splitting the data

```
x = df.drop("Class", axis=1)
y = df["Class"]
```

- X = all the columns that help us predict fraud
- y = the answer column, which is Class

```
x_train, x_test, y_train, y_test = train_test_split(
    x, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Take X and y and split them into 2 groups:

- X_train → input data for learning
- X_test → input data for testing
- y_train → correct answers for training data
- y_test → correct answers for testing data

Training & Prediction

```
model = DecisionTreeClassifier(random_state=42)

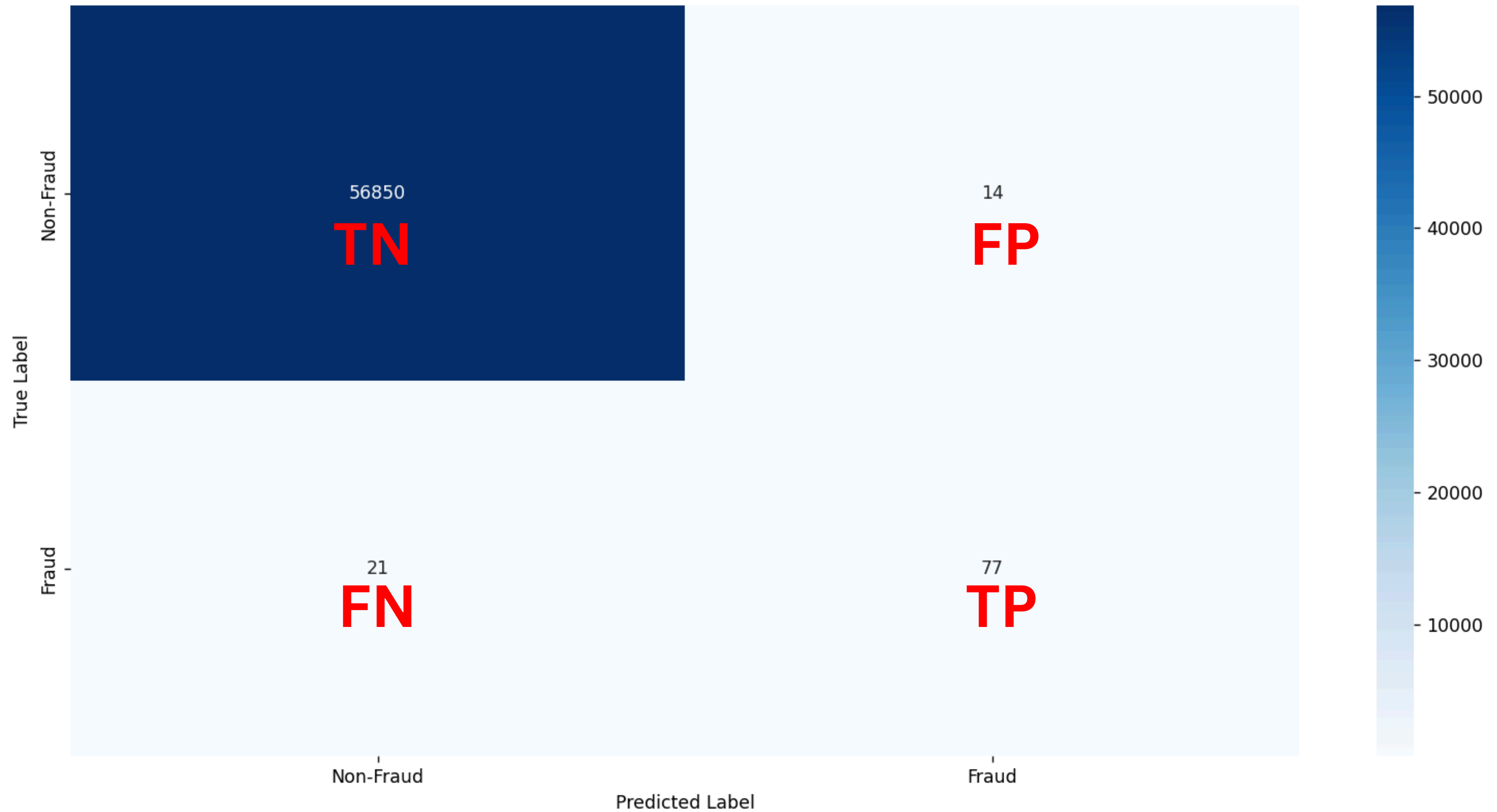
model = DecisionTreeClassifier(random_state=42, max_depth=4)
model.fit(x_train, y_train)

y_pred = model.predict(x_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Accuracy was : 99.94%

for fraud detection, accuracy alone is not enough.
so let's **visualize the confusion matrix**

Confusion Matrix



Credit Card Fraud Detection

Recall that we split 20% of the data to test the model (N=284,807) which is **56,962** rows

$$TN+TP= 56,850+77 = 56,927$$

So the accuracy is: $56,927/56962 = \mathbf{99.94\%}$

However... There are 98 total frauds in our test data, we predicted 77 correctly as fraud and predicted 21 incorrectly as non-fraud

Fraud Recall = $(TP / TP+FN) = 77/77+21 = 77/98 = \mathbf{78.6\%}$

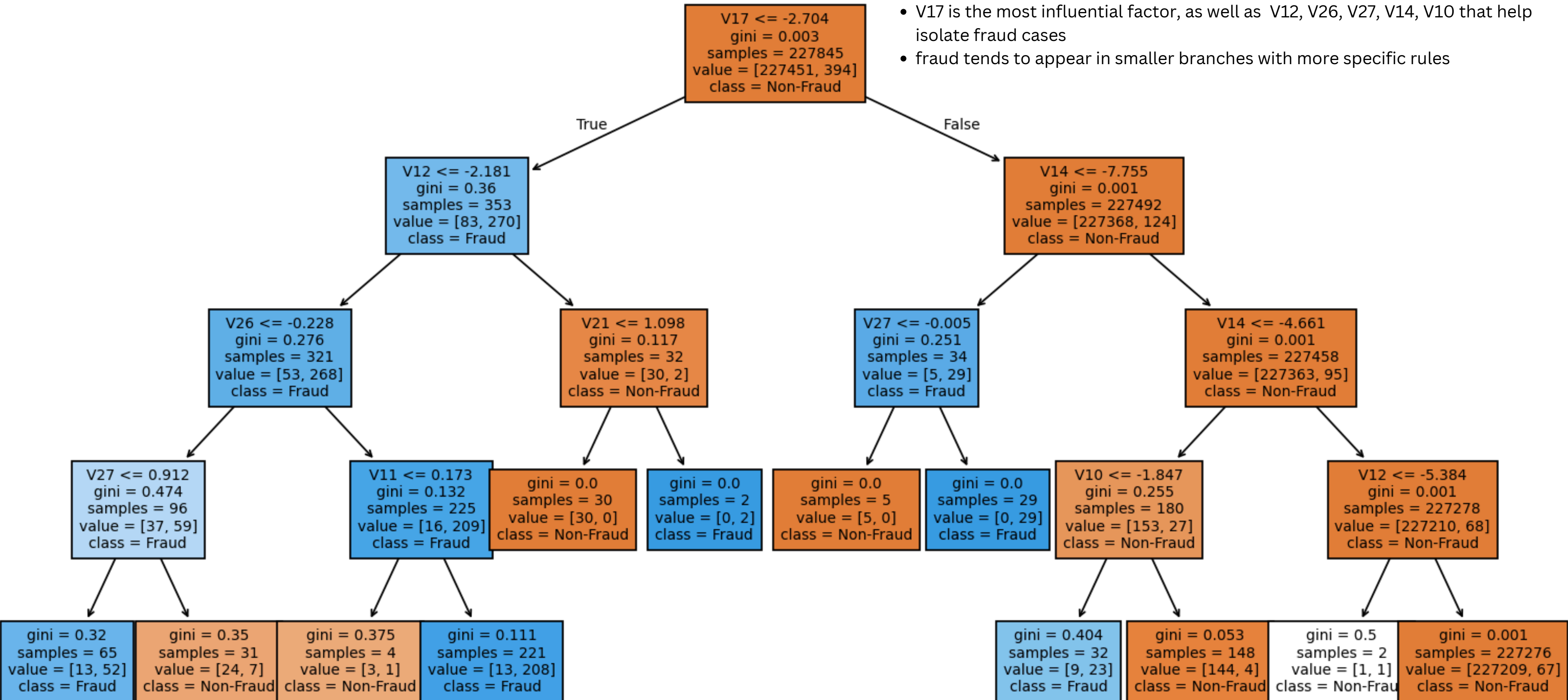
So the next question is:

Do we prefer fewer false alarms (**FP**), or do we prefer catching more frauds even if that means flagging more normal transactions?

In fraud detection, having more false positive is better than having less false negatives - Because if a transaction is labeled as a fraud while in reality it's not, we can reach out to authorities to reverse that label.

Decision Tree

- most transactions are non-fraud
- V17 is the most influential factor, as well as V12, V26, V27, V14, V10 that help isolate fraud cases
- fraud tends to appear in smaller branches with more specific rules



Credit Card Fraud Detection

Improving the model by: limiting tree depth to 4 branches to reduce overfitting, balancing class weights, and setting a minimum number of samples needed before a node is allowed to split.

```
model = DecisionTreeClassifier(  
    max_depth=4,  
    min_samples_split=10,  
    class_weight="balanced",  
    random_state=42  
)
```

Confusion Matrix



Credit Card Fraud Detection

Fraud Recall = $(TP / TP+FN) = 86/12+86 = 86/98 = 87.8\%$

Accuracy = **95.17%**

The model improved its ability to detect fraud. It caught 86 out of 98 fraud cases instead of 77, increasing fraud recall from 78.6% to 87.8%. However, this improvement came at the cost of many more false positives, meaning more normal transactions were incorrectly flagged as fraudulent and reducing the accuracy.

Conclusion: We used a Decision Tree classifier to detect fraudulent credit card transactions. Since the dataset was highly imbalanced, accuracy alone was not enough, so we focused on fraud recall. The model was able to learn decision rules and detect part of the fraudulent transactions, but performance depends strongly on handling imbalance and tuning the tree.

Dataset source: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>